

操作系统

何柯

2025 年 12 月 30 日

目录

1	内存管理	2
1.1	基础概念	2
1.2	内存分配方案	4
1.2.1	连续分配	4
1.2.2	离散分配	6
1.3	虚拟内存技术	8
1.4	优化技术	11

1 内存管理

1.1 基础概念

- 逻辑地址：这是程序视角下的地址，也称为相对地址，程序通常认为自己是地址 0 开始存放的
- 物理地址：这是硬件实现视角下的地址，也就是数据在内存条上实际存储的绝对位置，如果程序的基址（起始位置）是 1000，逻辑地址 500 对应的物理地址实际上是 1500
- 重定位技术：
 1. 静态重定位 (Static Relocation)
 - (a) 原理：编译器或加载器直接将程序中的指令地址修改为绝对物理地址；软件完成。加载器直接修改程序指令中的地址数据，把它变成绝对地址
 - (b) 缺点：程序一旦加载到内存，就不能移动；不支持多道程序运行
 2. 动态重定位 (Dynamic Relocation)
 - (a) 原理：在程序运行时进行地址转换；硬件支持。CPU 在执行每条指令时，自动加上基址寄存器的值
 - (b) 硬件支持：依赖于重定位寄存器（基址寄存器）
 - (c) 计算公式：
$$\text{物理地址} = \text{逻辑地址} + \text{基址寄存器的值}$$
 - (d) 优点：程序可以加载到内存的任意位置；运行时可以在内存中移动
- 内存保护 (Memory Protection)
 1. 使用界限寄存器 (Limit Register) 防止越界访问
 2. 检查条件：逻辑地址 < 界限寄存器值

3. 若不满足，触发越界异常；若满足，加上基址寄存器的值形成物理地址

- 覆盖与交换技术

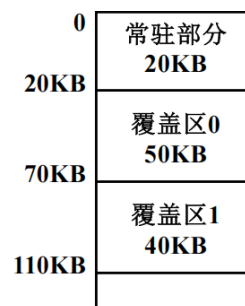
1. 覆盖技术 (Overlay)

- (a) 核心场景：针对单个程序。当程序代码量大于物理内存时的解决方案

- (b) 基本原理：

- i. 程序员手动切分程序为若干功能模块
- ii. 常驻区：存放核心代码，始终在内存中
- iii. 覆盖区：存放互斥模块（不同时执行），共用同一内存区域

- iv. 图解：



- (c) 例子：程序总大小 190KB，内存 110KB

- i. 常驻区：A 模块 (20KB)
- ii. 覆盖区 0：B (50KB) 和 C (30KB) 互斥
- iii. 覆盖区 1：D、E、F 中最大的 E (40KB)
- iv. 总需求：20 + 50 + 40 = 110KB

- (d) 优点：解决大程序在小内存上运行

- (e) 缺点：程序员必须手动规划模块调用关系，难度大

2. 交换技术 (Swapping)

- (a) 核心场景：针对多个程序（多道程序环境）。当内存里挤满了进程，新的进程进不来，或者被阻塞的进程占着茅坑不拉屎时，怎么办

- (b) 基本原理：操作系统充当“调度员”，把暂时不用的进程整个搬到硬盘（外存）里，腾出地方给急需运行的进程
- i. 换出 (Swap Out)：进程从内存移到硬盘对换区
 - ii. 换入 (Swap In)：进程从硬盘搬回内存运行
- (c) 特点：透明性强，支持多道程序，整体交换
- (d) 缺点：硬盘 I/O 开销大，性能较低

1.2 内存分配方案

1.2.1 连续分配

1. 固定分区分配 (Fixed Partitioning):

- 原理：将内存划分为若干固定大小的分区，每个分区可以容纳一个进程
- 优点：实现简单，易于管理
- 缺点：内存利用率低，可能产生大量碎片；数量固定；大小死板

2. 动态分区分配 (Dynamic Partitioning):

按程序需求分配不预先划分分区大小，存储结构有空闲分区表和空闲分区链表两种示意图如下：



图 1: 内存布局图

分区号	大小	起始地址
1	8KB	24KB
2	12KB	128KB
3	8KB	248KB
4	...	...
5	...	...

图 2: 空闲分区表

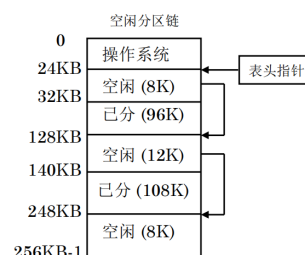


图 3: 空闲分区链

但是动态分区方案依然存在一个问题那就是外部碎片问题 (外部碎片是指内存中存在许多小的空闲分区，这些分区的总量虽然足以满足一

一个新作业的申请要求，但由于它们在物理位置上是不连续的（被已分配的作业隔开了），导致无法将该作业装入内存) 有如下两种解决方案:

(a) 紧凑化 (Compaction): 移动所有进程-> 开销极大

(b) 分配算法优化:

- 首次适应算法 (First Fit): 从头开始查找第一个满足要求的空闲分区 (特点: 优先利用内存低地址端, 高地址端有大空闲区。但低地址端有许多小空闲分区时会增加查找开销)
- 循环首次适应算法 (Next Fit): 从上次分配结束的位置开始查找 (特点: 使存储空间的利用更加均衡, 但会使系统缺乏大的空闲分区)
- 最佳适应算法 (Best Fit): 查找所有满足要求的空闲分区, 选择最小的一个 (特点: 减少内存浪费, 但会产生大量小的不可用碎片)
- 最差适应算法 (Worst Fit): 查找所有满足要求的空闲分区, 选择最大的一个 (特点: 剩下的空闲区比较大, 但当大作业到来时, 其存储空间的申请往往得不到满足)

对于算法的衡量好坏的标准: 对于某一个作业序列来说, 若某种分配算法能将该作业序列中所有作业安置完毕, 则称该分配算法对这一作业序列合适, 否则称为不合适

3. 伙伴系统 (Buddy System): 采用伙伴算法对空闲内存进行管理该方法通过不断对分大的空闲存储块来获得小的空闲存储块。当内存块释放时, 应尽可能合并空闲块。基本原理:

- 内存块大小均为 2^k 的形式
- 分配时, 找到最小的满足需求的 2^k 块
- 若找到的块大于需求, 则不断对半拆分, 直到接近需求大小
- 释放时, 检查相邻伙伴块 (大小相同) 是否空闲, 若是则合并

优缺点：

- 优点：回收效率高：合并算法简单快速；灵活性：比固定分区更灵活，且避免了动态分区中复杂的“紧凑”操作
- 缺点：内部碎片：由于分配块必须是 2 的幂次，若申请 65KB 需分配 128KB，会产生明显的内存浪费；拆分/合并开销：频繁的分配与回收会导致多次拆分与合并操作

例子：假设初始内存为 1MB，作业申请序列为：A(200K)，B(120K)。

- 分配 A(200K)：分配 256K 块，剩余 768K
- 分配 B(120K)：分配 128K 块，剩余 640K
- 释放 A：释放 256K 块，检查伙伴，无法合并

1.2.2 离散分配

1. 分页管理 (Paging):

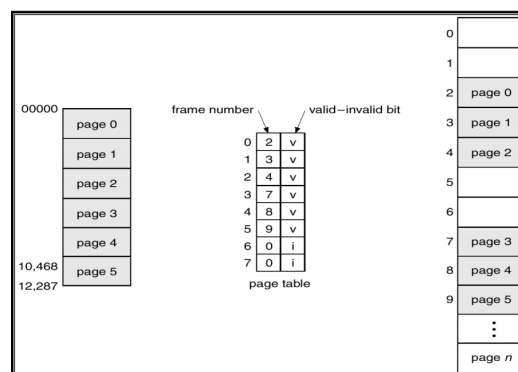


图 4: 分页管理示意图

页表 (如图 4)：逻辑页面-> 物理块页面-> 页框 (就是将进程划分为若干个固定大小的页，每页大小通常为 4KB，然后将这些页映射到物理内存中的页框)

计算地址：

$$\text{物理地址} = \text{页框号} \times \text{页大小} + \text{页内偏移}$$

快表：并行查找缓存最近使用的页表项，提高地址转换速度

表 1: 页表与快表比较

特性	页表 (Page Table)	快表 (TLB)
位置	主内存 (RAM)	CPU 内部 (MMU)
介质	DRAM (普通存储器)	SRAM (联想存储器)
容量	大，存放全部页表项	小，通常只有 64-1024 项
访问速度	慢	极快
查找方式	索引查找	并行比较查找

有效时间计算：给出一个例子吧：假设内存一次存取时间是 m ，联想寄存器的查找时间是 n ，命中率为 p 那么，有效存取时间 EAT 为：

$$EAT = (n + m) \times p + (n + 2m) \times (1 - p) = n + m + (1 - p) \times m$$

解释：命中时需要查找快表和访问内存，未命中时需要查找快表、访问页表和访问内存两次

不同的分页管理方式：

- 多级页表：将页表划分为多级结构，减少内存占用

例子：假设逻辑地址为 32 位，页面大小为 4KB (2 的 12 次方)，则页内偏移占 12 位，剩余 20 位用于页号。

页面大小 $\rightarrow 4KB = 2^{12}$ bytes $\rightarrow$ 页内偏移 12 位 $\rightarrow$ 剩余 20 位用于页号

将页号划分为两级：高 10 位作为一级页表索引，低 10 位作为二级页表索引 (因为每个页表项长度是 4 字节，所以每个页表可以包含 2^{10} 个页表项)

- 哈希页表：使用哈希函数映射逻辑页号到页表项，适用于大地址空间 (哈希函数 $\rightarrow$ 逻辑页号 (哈希链表中顺序查找) $\rightarrow$ 物理块号)
例子：假设逻辑页号为 **12345**，哈希表大小为 **1024**，哈希函数为 $h(x) = x \bmod 1024$ ，则 $h(12345) = 12345 \bmod 1024 = 57$ ，查找哈希表索引 **57** 处的链表以找到对应的页表项。 **57** $\rightarrow$ (**12345**, **5**) $\rightarrow$ (**6789**, **12**) $\rightarrow$...

- 反向页表：直接按序查找物理块对应的逻辑页，节省内存

2. 分段管理 (Segmentation): 段是逻辑划分的结构，按程序需求分配不预先划分分区大小，存储结构有段表，每个段有段号、段基址、段界限等信息

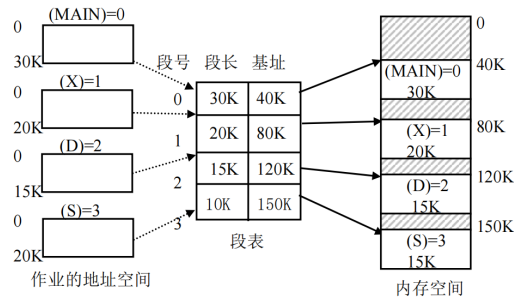


图 5: 分段管理示意图

3. 段页式管理 (Segmented Paging): 就是先用分段管理将程序划分为若干段，然后对每个段再进行分页管理

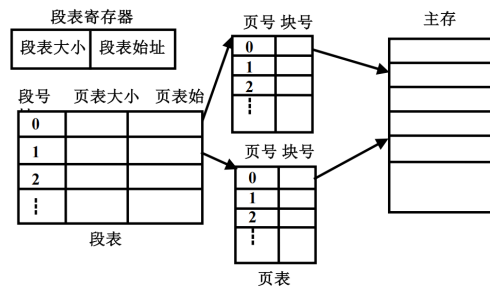


图 6: 段页式管理示意图

1.3 虚拟内存技术

虚拟内存技术: 将用户逻辑内存与物理内存分开, 使得程序可以使用比实际物理内存更大的地址空间

局部性原理: 程序在执行过程中, 在较短的一段时间内, 所执行的指令地址和操作数地址往往集中在一定的存储区域内

- 时间局部性：最近访问过的数据和指令在不久的将来可能会被再次访问（循环结构 → 带来时间局部性。）
- 空间局部性：与最近访问过的数据和指令地址相近的数据和指令在不久的将来可能会被访问（顺序执行 → 带来空间局部性。）

虚拟存储器的实现方法：

- 请求分页 (Demand Paging): 只有在页面被访问时才加载到内存

(a) 扩充后的页表结构:

- 有效位 (Valid Bit): 指示该页是否在内存中
- 修改位 (Dirty Bit): 指示该页是否被修改过
- 访问位 (Accessed Bit): 指示该页是否被访问过
- 页号和物理块号
- 外存位置指针: 指向该页在外存中的位置

(b) 页错误:

i. 发生条件: 访问的页不在内存中 (有效位为 0)

ii. 处理步骤:

- A. 检查页表以确定引用是否合法
- B. 引用非法则终止，有效但不在内存则调入
- C. 找到一个空闲帧
- D. 把页换入页框
- E. 修改页表，使其有
- F. 重启指令

(c) 缺页中断：就是处理页错误的过程

(d) 有效访问时间 (EAT) 计算: 内存的读写周期为 t ，缺页中断服务时间为 t_1 （包含读入缺页、页表更新、快表更新时间）快表的命中率为 α ，缺页中断率为 f ，快表访问时间为 ε ，则有效存取时间可表示为

$$EAT = \alpha \times (t + \varepsilon) + (1 - \alpha) \times [(1 - f) \times 2(t + \varepsilon) + f \times (t_1 + 2(t + \varepsilon))]$$

为什么是 $2(t + \varepsilon)$ 呢？因为未命中快表时，需要先访问页表（一次内存访问），然后再访问实际数据页（第二次内存访问），对于快表则是访问+修改

(e) 页面置换算法:

- FIFO: 先入先出，缺点：可能会淘汰掉经常使用的页面
- OPT: 最佳置换，缺点：需要预知未来访问情况，实际难以实现
- LRU: 最近最少使用，优点：较好地利用了局部性原理，缺点：实现复杂，需要维护访问时间信息
- LRU 近似算法: 使用访问位，定期清除访问位，选择未被访问的页面进行置换
- CLOCK: 时钟算法，使用一个指针循环扫描页面，选择访问位为 0 的页面进行置换
- 改进时钟算法: 结合修改位，优先选择访问位和修改位都为 0 的页面进行置换
- LFU: 最不经常使用，选择访问次数最少的页面进行置换

(f) 页面分配算法

- i. 平均分配：将物理内存均匀分配给所有进程
- ii. 按比例分配：根据进程大小按比例分配内存
- iii. 按优先级分配：分为两部分，先按比例分配，再根据进程优先级分配内存

(g) 抖动 (Thrashing): 如果运行进程的大部分时间都用于页面的换入/换出，而几乎不能完成任何有效的工作，则称此进程处于抖动状态。抖动又称为颠簸、颤动

产生原因:

- 进程分配的物理块太少
- 置换算法选择不当
- 全局置换使抖动传播

(h) 工作集模型 (Working Set Model):

- 定义：在某一时间段内，进程实际使用的页面集合
- 工作集窗口 (Δ)：定义为最近 Δ 时间内访问的页面集合
- 工作集大小 (WSS)：工作集窗口内的页面数量
- 目标：保持进程的工作集在内存中，减少缺页率

举个例子：观察窗口 (Δ)：管理员盯着你看了 10 分钟（这就是工作集窗口）；工作集 (WSS)：他发现这 10 分钟里，你反反复复翻阅的其实只有《数学》、《物理》、《化学》这 3 本书
决策：虽然你有 100 本参考书，但在当前阶段，只要给你书桌上腾出 3 个位置，你就能顺畅工作了

- 请求分段 (Demand Segmentation): 只有在段被访问时才加载到内存

优点：

- 有利于动态增长
- 允许按段进行编译
- 有利于共享和保护

1.4 优化技术

写时复制 (**COW**)：只要没人要修改数据，大家就共享同一份；一旦有人要修改，再给他单独复制一份

这个过程利用了“只读”标记和“欺骗”手段：

1. 初始状态：父子进程共享同一页，页表项标记为“只读”
2. 修改请求：当某个进程尝试写入该页时，CPU 发现该页面是“只读”的，于是报错（触发异常/中断）
3. 处理异常：操作系统捕获这个中断，发现是因为“写时拷贝”机制引起的，于是它说：“好吧，既然你要改，那我现在给你复制一份。”
4. 结果：现在，进程 1 拥有了自己的新页面，进程 2 还在用旧页面。只有被修改的那一页发生了复制，其他没改的页依然共享。